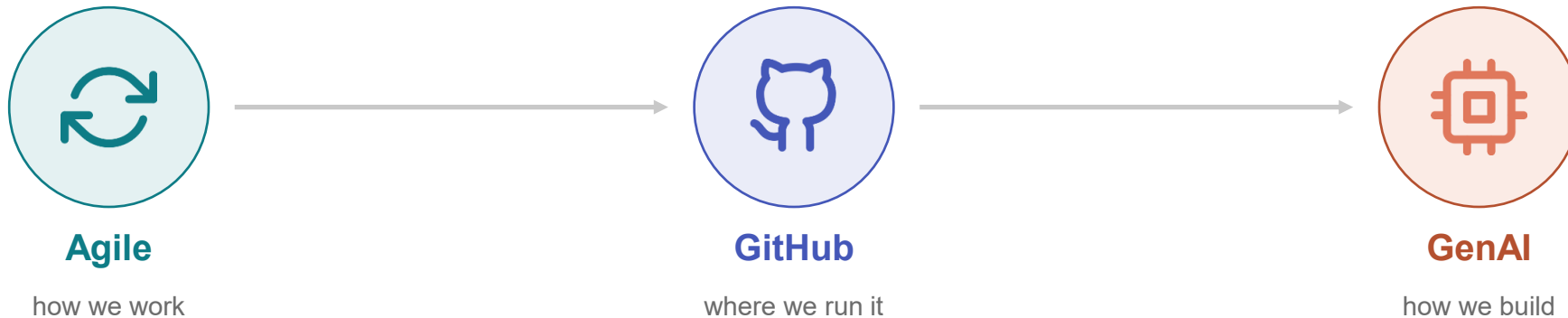


1. Getting Started with Sprint 1



What this deck covers



1

Agile essentials

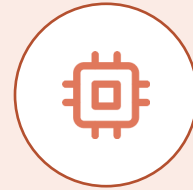
The handful of concepts a two-person team actually needs — stories, backlog, sprints, done, and the rituals.



2

Running sprints on GitHub

How Issues, Projects, and Pull Requests map directly onto the Agile workflow you just learned.



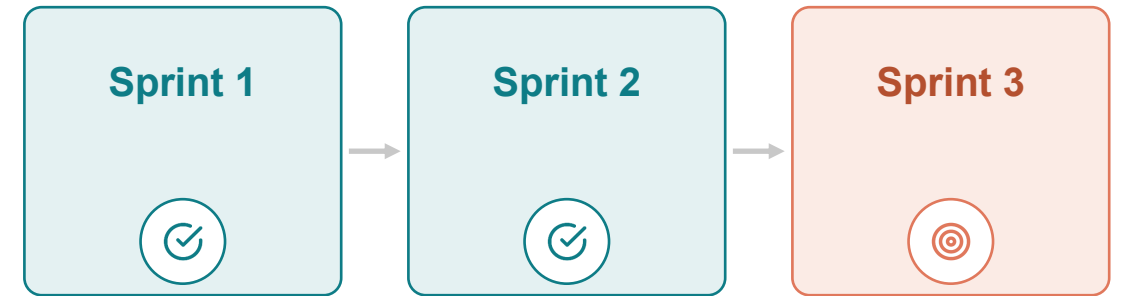
3

Setting up Sprint 1 with GenAI

Where AI tools help, the golden rule of ownership, prompting well, and your setup checklist.

The project, in one slide

- You work in pairs.
- You build a working product prototype — not a report.
- GenAI coding tools support you the whole way.
- Each sprint ends with something that runs, plus a demo and a retro.



Each iteration = a working increment

01

Agile essentials

The concepts a two-person team actually needs



The core idea: small, working increments

Big-bang

Plan everything → build for weeks → hope it works at the end

Risk piles up; problems surface too late.



Agile



Each loop delivers something that runs — you learn and adjust every two weeks.

“Done and working” beats “lots of code, nothing runs.”

User stories: features from the user's point of view



As a **[type of user]**, I want **[goal]**, so that **[benefit]**.

Example

As a student, I want to filter events by category, so that I can find relevant ones quickly.

Acceptance criteria

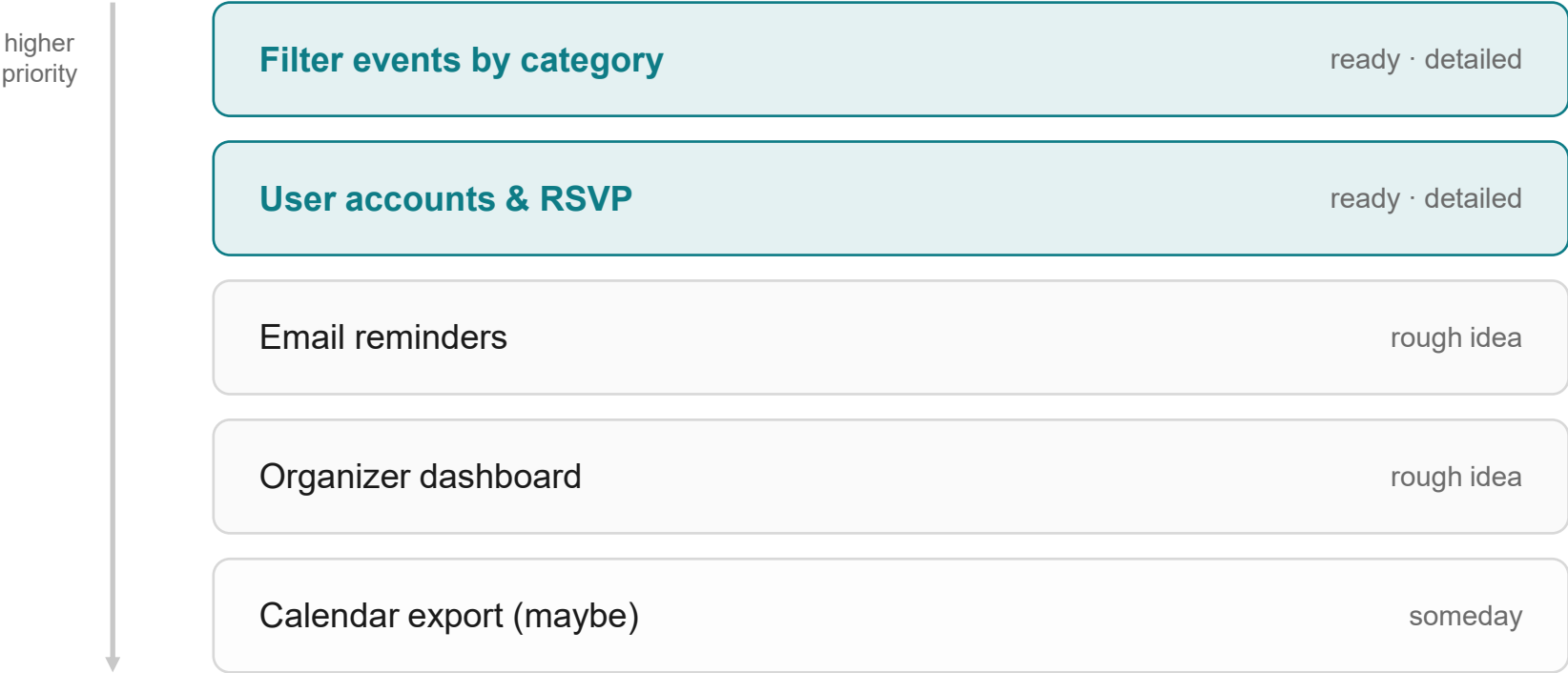
- A category filter is visible on the events page.
- Selecting a category shows only matching events.
- Clearing the filter restores the full list.

Acceptance criteria turn a vague idea into something you can call “done”.

The backlog: a living, prioritized list



Everything the product might need, ordered by priority. You pull from the top each sprint; the rest can change as you learn.



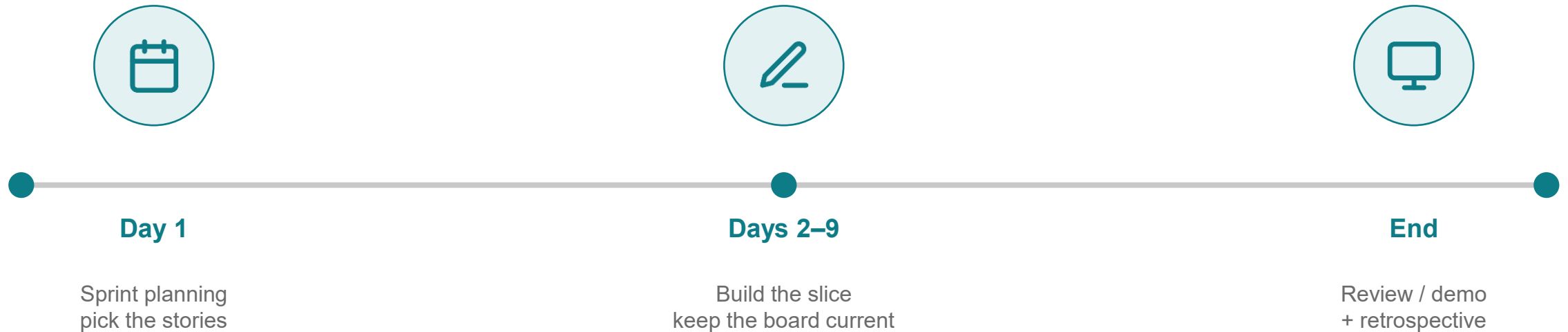
Top of the list

Small, well-understood, ready to build now.

Bottom of the list

Fuzzy and low-priority — fine to leave vague until it rises.

The sprint: a fixed two-week timebox



The deadline is fixed.

Two weeks is two weeks — you don't extend the sprint.

The scope flexes.

If you took on too much, cut scope — don't cut quality.

Definition of Done: what “finished” really means



A shared checklist your team agrees on. A story is not “done” until every box is ticked.



Merged into the main branch



Runs without errors



Meets the story’s acceptance criteria



A basic test passes



Reviewed by your teammate



Without a shared Definition of Done, “done” quietly means “works on my laptop.”

Three rituals you will actually use



Sprint planning

Start of sprint

Pick the stories for the next two weeks and break them into tasks.



Review / demo

End of sprint

Show the working increment. If it can't be demoed, it isn't done.



Retrospective

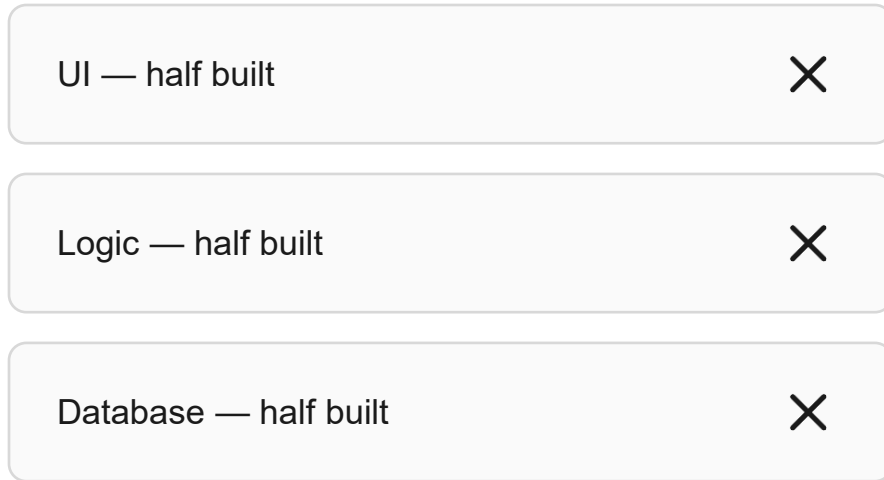
End of sprint

A short, honest look at what to change next sprint. Where most learning happens.

Don't skip the retro — even 15 minutes pays off across three sprints.

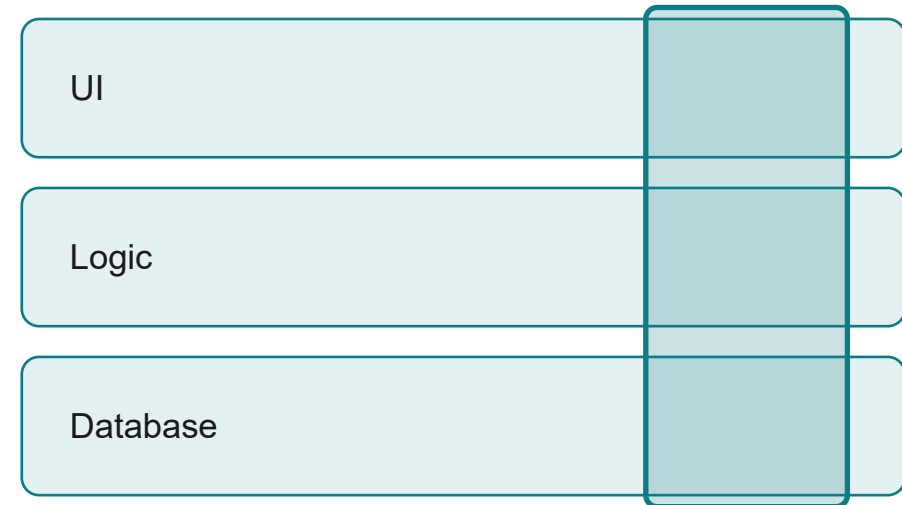
Build a vertical slice, not layers

Tempting but wrong



Three half-finished layers = nothing a user can do.

Do this instead



One feature, all the way through. ✓

Sprint-1 example: “Add an event and see it appear in the list” — UI → API → database, working.

What to skip (for a two-person team)



Keep

- User stories with acceptance criteria
- A simple prioritized backlog
- Two-week sprints with a clear goal
- A Definition of Done
- Planning, demo, and retro



Skip (for now)

- Daily stand-up meetings
- Story-point estimation
- Formal Product Owner / Scrum Master roles
- Burndown charts and velocity tracking

These exist in bigger teams — two people talking regularly don't need the ceremony.

02

Running sprints on GitHub

Issues, Projects, and Pull Requests



Your GitHub toolkit maps onto Agile



Repository



The home for your code and history



Issues



Your user stories and tasks



Projects



Your sprint board (backlog → done)



Pull Requests



Review and merge — your Definition of Done

Issues are your stories and tasks



Filter events by category

As a student, I want to filter events by category, so that I can find relevant ones quickly.

Acceptance criteria

- ☐ Category filter visible on the events page
- ☐ Selecting a category shows only matches
- ☐ Clearing the filter restores the full list

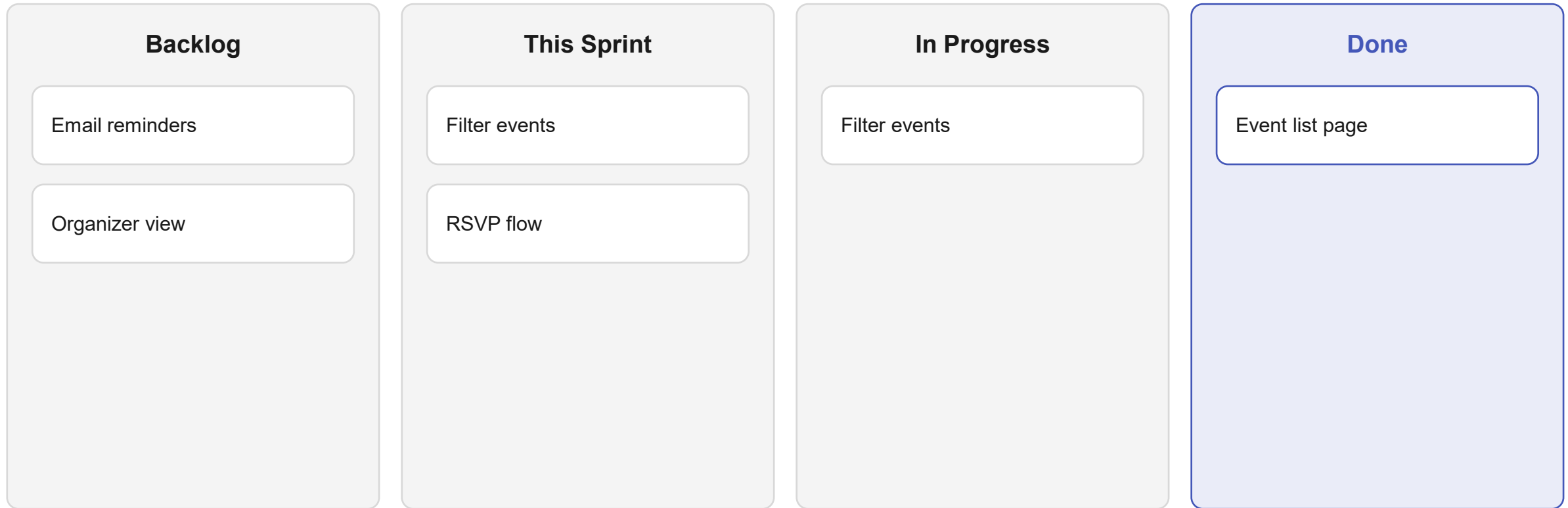
sprint-1

feature

Make it a habit

- Write the story in the issue body.
- List acceptance criteria as checkboxes.
- Add labels (e.g. sprint-1, feature, bug).
- Assign an owner so work isn't doubled.
- Close it from the pull request that finishes it.

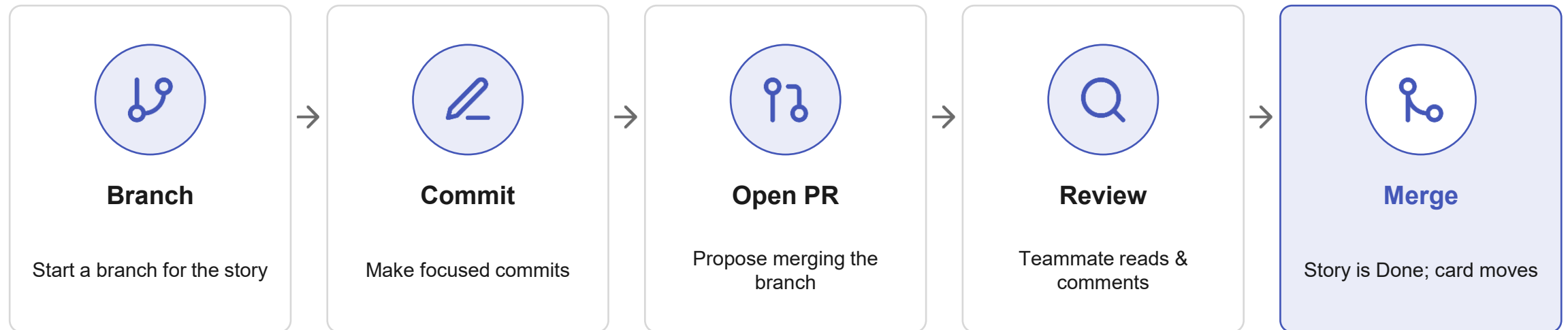
Projects is your sprint board



Cards move left → right as work progresses

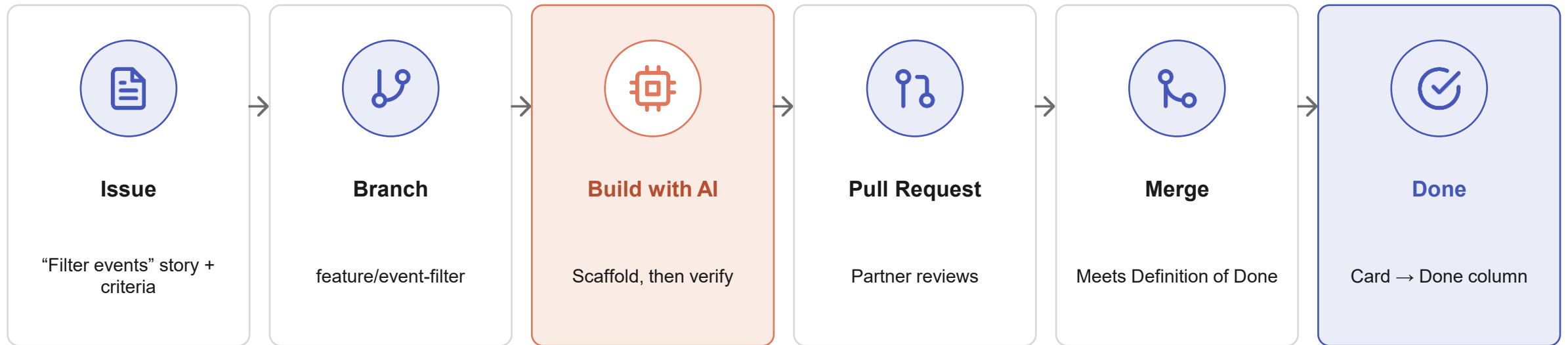
A stale board is worse than none — it hides where the work really stands. Update it before you stop each session.

Pull Requests: how work becomes “Done”



Even in a pair, review each other's pull requests. It's your safety net — especially for code a GenAI tool generated that neither of you wrote by hand.

One feature, end to end on GitHub



That full loop is one vertical slice — and it's exactly what Sprint 1 asks for.

03



Setting up Sprint 1 with GenAI tools

Where AI helps — and how to stay in control

The GenAI developer toolkit

The course doesn't mandate one tool — pick the right kind for the task.



In-editor assistants

e.g. GitHub Copilot, Cursor

Autocomplete and inline chat right in your editor. Great for writing code in flow and small edits.



Agentic / CLI tools

e.g. Claude Code

Work across many files, run commands, and carry out multi-step tasks you describe in plain language.



Chat assistants

general chat LLMs

Explain unfamiliar code, plan an approach, or talk through a bug — outside the editor.

Where GenAI earns its keep



Scaffolding & boilerplate

Project skeletons, CRUD endpoints, config.



Generating tests

Draft unit tests, then you review coverage.



Unfamiliar APIs

Show usage of a library you don't know yet.



Debugging errors

Paste an error; get likely causes and fixes.



Refactoring

Rename, extract, simplify — then run tests.



Docs & commit messages

Draft READMEs, docstrings, clear commits.

The golden rule: you own every line



If you can't explain a piece of code you submitted, you're not done with it.

What this means in practice

- Read generated code before you accept it.
- Be able to walk through why it works.
- You'll be assessed on understanding, not just output.
- Rejecting or rewriting AI output is a valid, expected choice.



Over-reliance, in one story

A team pastes a generated function that “just works.” In the demo it throws an error on an empty input — and neither student can debug it, because neither understood it. The slice fails live.

Prompt like an engineer, not a wisher

✗ Vague

“Make a login page.”

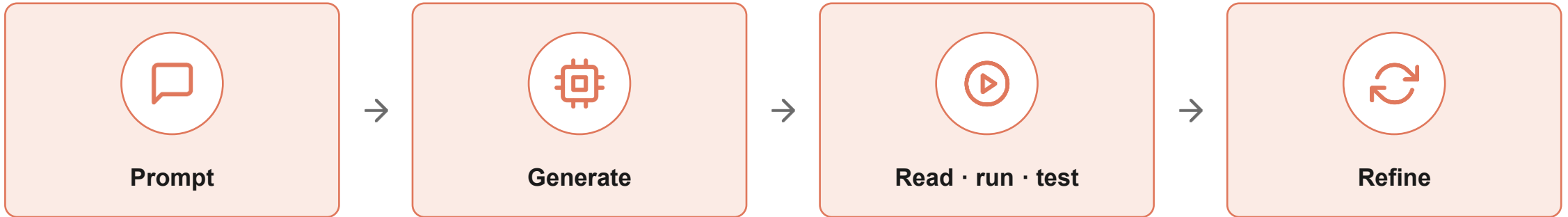
Good prompts carry:

- Context — the stack and existing components
- Constraints — inputs, validation, endpoints
- Examples — patterns to follow
- A test request — so you can verify it

✓ Specific

“Generate a React login form with email + password fields, client-side validation for empty fields and email format, and a submit handler that calls POST /api/login. Reuse our existing Button component. Include a Jest test for the validation.”

Trust, but verify



Repeat until it's right — refinement is normal, not failure.

Watch for

- Hallucinated APIs — functions or libraries that don't exist
- Code that runs but ignores an edge case (empty input, nulls)
- Confident explanations that are subtly wrong

Example

Generated code calls `dateFns.formatPretty()` — which doesn't exist. Running it (or a quick test) catches the hallucinated API in seconds.

Keep a short AI usage log



A few lines per session. It feeds your retro, builds an evaluation mindset, and guards against silent over-reliance.

Task	Tool	Helped?	What I had to fix
Scaffold event API	Copilot	Yes	Added validation it missed
Explain GTFS parsing	Chat LLM	Partly	Verified against real docs
Write filter tests	Claude Code	Yes	Added an empty-list case

This same habit is what turns you into a critical user of these tools — a core skill of the course.

Sprint 1 setup checklist



Create the repo — add both teammates



Agree on your tech stack



Get the environment running



Make a “hello world” run end-to-end



Install & sign in to your GenAI tool



Create the Project board (4 columns)



Write your first 3–6 issues as stories



Pick the ONE vertical slice for Sprint 1

Your Sprint 1 goal



One vertical slice that works end-to-end.

One feature, through every layer — proving your stack, your workflow, and your team are ready.

A working demo

A 15-minute retro

A groomed backlog for Sprint 2

Go deeper — all free



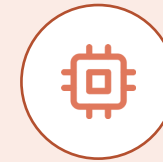
Agile & Scrum

- Agile Manifesto — agilemanifesto.org
- The Scrum Guide — scrumguides.org
- Atlassian Agile Coach — atlassian.com/agile
- User stories — mountaingoatsoftware.com



GitHub workflow

- Issues — docs.github.com/en/issues
- Projects — [docs.github.com/en/issues \(Projects\)](https://docs.github.com/en/issues/projects)
- Hands-on — skills.github.com



Prompting & GenAI

- Prompt Engineering Guide — promptingguide.ai
- GitHub Copilot docs — docs.github.com/copilot